COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 11-15, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 8.1 **Sorting Algorithms Part 2**

1. PROCEDURE OUTPUT

What procedures did you perform?

Code

```
#include <iostream>
#include <cstdlib>
#include <ctime>

const int maxSize = 100;

void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    int unsortedArr[maxSize];
    srand(time(0));

    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 100;
    }

    std::cout << "Original Unsorted Array (100
Elements):\n";
    displayArr(unsortedArr);
```

```
    return 0;
}
```

Output

```
Original Unsorted Array (100 Elements):
35 73 96 59 80 91 86 43 58 58 7 94 27 84 42 97 23 47 85
 19 56 88 70 1 2 94 85 76 16 94 97 89 67 85 5 15 74 13
28 2 21 19 4 87 13 28 3 92 80 74 0 5 6 30 57 15 96 40 4
8 60 68 23 23 64 88 37 59 10 61 85 25 49 63 79 4 53 40
71 37 61 6 50 60 11 73 30 6 10 24 29 39 86 51 91 95 9 5
7 83 55 76
```

Observations

Using the srand function with time(0) and filling a 100-element array with rand()%100 provides an unpredictable and non-deterministic set of numbers including duplicates and different distributions. Due to the fact that all numbers are unordered, the array creates an unbiased benchmark to test how well the Shell Sort, Merge Sort, and Quick Sort algorithms perform in handling large distances and recursion.

Table 8-1. Array of Values for Sort Algorithm Testing

Code

```
//B.1 Shell Sort
template <typename T>
void shellSort(T arr[], int n) {
    // Start with a large gap, then reduce the gap
    for (int gap = n / 2; gap > 0; gap /= 2) {
        // Do a gapped insertion sort for this gap size
        for (int i = gap; i < n; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j
-= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

sortingAlgo.h code

```
//Table 8-2. Shell Sort Technique
    std::cout << "\nShell Sort Output:\n";
    shellSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
```

main.cpp

Output

```
Shell Sort Output:
0 1 4 6 6 7 7 7 8 9 10 10 11 11 13 14 14 16
 17 20 21 23 24 25 25 25 25 25 26 30 30 31
33 33 35 35 35 35 37 37 39 41 41 41 42 43 4
4 44 46 47 47 48 48 53 53 54 54 55 55 56 56
 57 61 61 62 62 62 64 65 65 66 66 66 68 70
71 72 73 76 77 77 78 79 79 80 81 81 83 85 8
5 86 87 88 89 90 92 92 94 96 98
```

Observations

Lab manual code for bubble sort uses two iterative structures that compare a single index i with all subsequent indices j, and swap the elements at once whenever an out-of-order arrangement is noticed. Although this approach moves all target elements to their rightful places within several iterations, looping through two nested loops for all 100 elements causes quadratic time complexity of O(n^2).

Table 8-2. Shell Sort Technique

Code

```
//B.2 Merge Sort
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
```

```
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

sortingAlgo.h code

```
// Table 7-3: Selection Sort
    std::cout << "Table 7-3: Selection Sort Output" <<
std::endl;
    selectionSort(arrSelection, maxSize);
    displayArr(arrSelection);
```

main.cpp

Output

```
Merge Sort Output:
0 3 3 3 3 4 4 4 5 5 6 7 7 8 8 9 13 13 14 14 15 18 18 21 22
 22 22 23 24 25 27 28 28 30 35 37 37 38 39 40 40 44 44 45
46 46 48 51 53 54 54 55 55 55 55 57 59 60 60 60 63 63 63 6
5 68 69 71 72 73 73 73 73 74 74 75 76 76 76 77 77 77 79 80
 81 83 84 84 85 85 85 86 87 90 96 97 97 97 98 98 99
```

Observations

Merge Sort is based on the principle of Divide and Conquer, whereby Merge Sort algorithm recursively divides the 100-element array until individual elements remain before reassembling the array using the process of merging in pairs. With division taking $\log_2(n)$ time and merging taking $O(n)$ time, Merge Sort algorithm ensures that there is always predictable time complexity of $O(n \log n)$ in all situations.

Table 8-3. Merge Sort Algorithm

Code

```cpp
//B.3 Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high]; // Last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

sortingAlgo.h code

```cpp
// Table 8-4. Quick Sort
    std::cout << "\nQuick Sort Output:\n";
    quickSort(unsortedArr, 0, maxSize - 1);
    displayArr(unsortedArr);
```

main.cpp

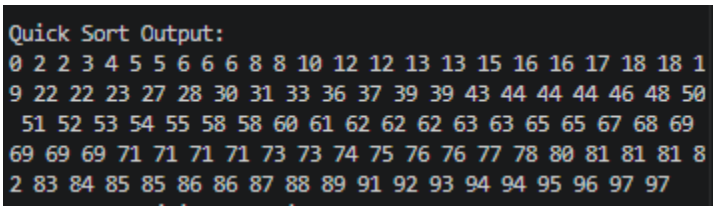
Output	
Observations	In quick sort, partitioning takes place within the same array by choosing one of the elements as a pivot element, where all elements smaller than the pivot are arranged to the left, while all elements greater than the pivot are moved to the right. After the position of the pivot becomes fixed, further sorting happens independently through recursive calls. It executes in-place with no use of additional storage, with a time complexity of $O(n \log n)$.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer:

Yes, the sorted sub-lists created after partitioning can certainly be sorted through other sorting algorithms. Once the partition() method assigns the pivot element its rightful place, it is ensured that all the elements on the left sub-list are smaller than or equal to the pivot while all the elements on the right sub-list are larger than or equal to the pivot. Since there are no further requirements to cross over from one side to another past the pivot, it can be sorted using any other sort since the two partitions will be completely independent of each other. The implementation of such a process in reality is that of C++ std::sort (Introsort), where insertion sort is used for small partitions.

Demonstration Example:

Initial array: {24, 9, 29, 14, 45, 18}

Step 1 (Partition around pivot = 18):

Rearranging elements around 18 leaves:

Left Partition: {9, 14} | Pivot: [18] | Right Partition: {45, 29, 24}

Step 2 (Sort partitions using another algorithm):

Apply **Insertion Sort** to the Left Partition {9, 14} -> {9, 14}.

Apply **Selection Sort** to the Right Partition {45, 29, 24} -> {24, 29, 45}.

Step 3 (Recombine):

Connecting Left + Pivot + Right yields the fully sorted array: {9, 14, 18, 24, 29, 45}.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have $O(N \cdot \log N)$ for their time complexity?

Answer:

Fastest algorithm for this array: Quick Sort (or else **Insertion Sort** or **Shell Sort** if array size is considered).

Quick Sort will give fastest runtime for an unsorted, degenerate 15 element array. This is because it works in place with no use of temporary heap or buffer space (unlike Merge Sort). The inner comparison and swapping loops will have least overhead with efficient CPU cache access. Arrays of this size (N = 15) run in essentially zero overhead due to Insertion Sort and Shell Sort.

Why Merge Sort and Quick Sort achieve $O(N \log N)$ time complexity:

Both sorting methods depend on the **Divide and Conquer** strategy that separates the sorting process into two components from the structural point of view:

- **The $\log N$ Factor (Number of Tree Levels / Number of Divisions):**

  To split a collection of size N into equal parts multiple times results in building a balanced binary decomposition tree. It will take $\log_2 N$ number of steps to divide N down to one-element cases when $N = 1$.

- **The N Factor (Linear Work on Each Level):**

  In each level of decomposition tree all N elements have to be analyzed once:

  - In the case of **Merge Sort**, the comparison and merge of sub-arrays on each level will require linear number of operations equal to $O(N)$.
  - In the case of **Quick Sort**, the partitioning of the array by the pivot on each level will require the analysis of each element once, thus resulting in $O(N)$ number of

comparisons.

Multiplication of linear number of operations on each level (N) by the number of levels ($\log N$) will give overall complexity of $O(N \cdot \log N)$.

Conclusions: This lab activity provided hands-on experience about how Shell Sort, Merge Sort, and Quick Sort work and perform in C++. Running all three sorts on a random 100 element array helped me understand how these algorithms work around their $O(n^2)$ limitations. Shell Sort used optimized insertion sort, where distant elements were compared within decreasing gap intervals to pre-sort the array. In addition, Merge Sort and Quick Sort demonstrated the advantages of divide-and-conquer approach: Merge Sort ensured guaranteed $O(n \log n)$ running time through recursive dividing and merging of an array, while Quick Sort accomplished fast in-place sorting based on pivot element. The supplementary activity showed that sub-lists could be sorted individually using some other methods and mathematically confirmed the $O(n \log n)$ complexity by multiplying linear-level operations with logarithmic-level number of divisions. Overall, I did well in working with templates, manipulating dynamically allocated arrays during merging, and tracing recursive calls. The aspect that should be worked on next is implementing a pivot selection by median-of-three method in Quick Sort to avoid degradation in worst cases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.

Purpose of Use: **For code assistance in some parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an observation and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI firstly to understand the concepts of procedure part more and also improve**

the grammars of observations of it. Second, to help me understand the supplementary activity. It's because I need to have more clarity on what each items of the supplementary is to answer it precisely. Lastly, I used AI to improve my grammar and tone in forming my conclusion.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.